

Multiplicación de Matrices en C Paralelización con OpenMP

HPCG1-CE02

INTEGRANTES

David Santiago Lugo Cabrera
Jose Aroudo Junior de Asis Pinedo
Angie Carolina Vargas Villegas

1. Objetivo de la actividad

El objetivo de esta entrega fue analizar e informar el comportamiento de la **multiplicacion de matrices** en C, incluyendo:

- implementacion base serial,
- implementacion concurrente con OpenMP,
- caracterizacion de CPU y memoria,
- benchmark de desempeno,
- comparacion de resultados en dos equipos,
- y preparacion del documento final para sustentacion.

En esta version se reportan la base serial en `src/serial/multmat_serial.c` y la version concurrente en `src/openmp/multmat_openmp.c`.

2. Descripcion de la implementacion

La base serial realiza la multiplicacion de dos matrices cuadradas $N \times N$ generadas aleatoriamente. El flujo general es:

1. reservar memoria para `A`, `B` y `C`,
2. llenar `A` y `B` con valores aleatorios,
3. ejecutar el kernel serial de multiplicacion,
4. imprimir tiempos de llenado, multiplicacion y total.

La version OpenMP conserva el mismo resultado numerico, pero divide el trabajo por filas entre hilos. Antes de multiplicar, transpone `B` para mejorar la localidad espacial del acceso.

La medida principal del programa es el tiempo de multiplicacion, porque ese tramo concentra la mayor parte del costo computacional.

3. Caracterizacion de CPU

3.1 Complejidad algoritmica

El kernel serial usa tres bucles anidados:

- `i` recorre las filas de `A`,
- `j` recorre las columnas de `B`,
- `k` recorre la suma producto.

Eso produce una complejidad de $O(N^3)$ en tiempo.

3.2 Perfil CPU

El consumo de CPU está dominado por:

- operaciones enteras en el bucle interno,
- multiplicaciones y sumas acumuladas en `long long`,
- accesos repetidos a memoria dentro del kernel.

En la practica, el tiempo de CPU crece de forma cubica con el tamaño de la matriz, por lo que los casos pequenos sirven como linea base y los casos medianos/grandes muestran mejor la carga real del procesador.

3.3 Perfilado de CPU medido en esta maquina

El benchmark ejecutado en esta maquina con 10 corridas por configuracion arrojo estos tiempos promedio de multiplicacion:

Configuracion	N=100	N=500	N=1000	N=2000
Serial	0.459 ms	53.411 ms	413.872 ms	3443.264 ms
OpenMP-4	1.350 ms	17.160 ms	122.030 ms	964.856 ms
OpenMP-8	1.284 ms	12.869 ms	84.176 ms	636.942 ms

Interpretacion:

- Para `N=100`, la sobrecarga de OpenMP supera el beneficio.
- Desde `N=500`, OpenMP empieza a mostrar ventaja clara.
- En `N=1000` y `N=2000`, la version de 8 hilos es la mas rapida de esta maquina.

3.3 Medicion usada en el informe

El programa ya reporta el tiempo de multiplicacion en milisegundos. Para el informe se toman las corridas almacenadas en:

- [benchmark_summary.csv](./benchmark_summary.csv)

4. Caracterizacion de memoria

4.1 Uso de memoria por estructura

La implementacion serial reserva tres arreglos principales:

- `A`: `N x N` enteros,
- `B`: `N x N` enteros,
- `C`: `N x N` valores `long long`.

4.2 Formula de memoria

Si un `int` ocupa 4 bytes y un `long long` ocupa 8 bytes, el uso aproximado es:

\$\$

$$M(N) = 4N^2 + 4N^2 + 8N^2 = 16N^2 \text{ bytes}$$

\$\$

Esto equivale, aproximadamente, a:

- `N=100`: 160000 bytes, unos 0.15 MiB,
- `N=500`: 4000000 bytes, unos 3.81 MiB,

- `N=1000`: 16000000 bytes, unos 15.26 MiB,
- `N=2000`: 64000000 bytes, unos 61.04 MiB.

4.3 Interpretacion

La memoria crece de forma cuadratica con `N`, mientras que el tiempo crece de forma cubica. Por eso, en este problema el cuello principal termina siendo CPU y no solo capacidad de almacenamiento.

4.4 Perfilado de memoria en esta maquina

El consumo teorico estimado para las estructuras del programa es:

N	Memoria estimada
100	160000 bytes (~0.15 MiB)
500	4000000 bytes (~3.81 MiB)
1000	16000000 bytes (~15.26 MiB)
2000	64000000 bytes (~61.04 MiB)

En la practica, el consumo real del proceso depende del runtime, pero este calculo permite dimensionar el crecimiento del problema y justificar por que el benchmark de `N=2000` ya presiona con mas fuerza el subsistema de memoria.

5. Perfilado real de CPU y memoria

El perfilado se realizo ejecutando el binario y muestreando el proceso mientras corria. Se midieron estos indicadores:

- `Wall\_s`: tiempo total real de ejecucion,
- `CPU\_s`: tiempo acumulado de CPU del proceso,
- `PeakWorkingSet\_MB`: pico de memoria residente,
- `PeakPrivateMemory\_MB`: pico de memoria privada,
- `CPU\_to\_Wall`: relacion entre CPU y tiempo real, util para ver el uso efectivo de multiples nucleos.

Los datos provinieron de profile_summary.csv.

5.1 Perfilado de la base serial

N	Wall_s	CPU_s	Peak Working Set (MB)	Peak Private Memory (MB)	CPU/Wall
100	0.037696	0.000000	3.02	0.44	0.00
500	0.091642	0.046875	6.61	4.36	0.51

1000	0.464862	0.437500	18.86	15.82	0.94
2000	3.597797	3.515625	64.94	61.69	0.98

5.2 Perfilado de OpenMP con 4 hilos

N	Wall_s	CPU_s	Peak Working Set (MB)	Peak Private Memory (MB)	CPU/Wall
100	0.039136	0.000000	3.02	0.44	0.00
500	0.056573	0.062500	8.91	5.78	1.10
1000	0.178938	0.468750	23.89	20.13	2.62
2000	1.056642	3.750000	81.19	77.48	3.55

5.3 Perfilado de OpenMP con 8 hilos

N	Wall_s	CPU_s	Peak Working Set (MB)	Peak Private Memory (MB)	CPU/Wall
100	0.039834	0.000000	3.02	0.44	0.00
500	0.057444	0.125000	8.90	5.91	2.18
1000	0.146524	0.703125	24.02	20.27	4.80
2000	0.767008	5.109375	81.09	77.71	6.66

5.4 Lectura del perfilado

- La memoria privada pico sube con el tamaño de la matriz, como era esperable por la fórmula $16N^2$.
- La versión serial usa aproximadamente un núcleo completo cuando N es grande, porque CPU_s y $Wall_s$ casi coinciden.
- OpenMP aumenta CPU_s porque varios hilos consumen CPU en paralelo; por eso la razón $CPU/Wall$ crece con el número de hilos.
- El mayor pico de memoria aparece en $N=2000$, con alrededor de 61.69 MB en serial y 77.71 MB en OpenMP-8.

6. Metodología de benchmark

Para conservar comparabilidad se empleó la misma estrategia en todas las corridas:

- mismos ejecutables serial y OpenMP,
- mismos tamanos de matriz,
- 10 corridas por configuracion,
- compilacion con optimizacion de compilador `-O2` como base y `-fopenmp` para la version paralela.

Los tamanos evaluados fueron:

- `N = 100`
- `N = 500`
- `N = 1000`
- `N = 2000`

7. Resultados del benchmark en dos maquinas

La actividad requiere comparar en dos equipos. Se ejecuto el mismo procedimiento en ambas máquinas:

1. compilar con scripts/build.ps1,
2. ejecutar scripts/benchmark.ps1,
3. 10 corridas por configuracion,
4. registrar promedio, desviacion, minimo y maximo.

7.1 Maquina 1 - Resultados de Benchmark

7.1.1 Serial

N	Promedio (ms)	Desvio (ms)	Min (ms)	Max (ms)
100	0.459	0.013	0.442	0.485
500	53.411	1.057	51.357	54.818
1000	413.872	4.352	410.173	426.033
2000	3443.264	24.931	3408.558	3482.634

7.1.2 OpenMP 4 hilos

N	Promedio (ms)	Desvio (ms)	Min (ms)	Max (ms)
100	1.350	0.061	1.254	1.466
500	17.160	2.043	15.156	22.761

1000	122.030	7.868	110.307	137.591
2000	964.856	23.085	929.508	1012.020

7.1.3 OpenMP 8 hilos

N	Promedio (ms)	Desvio (ms)	Min (ms)	Max (ms)
100	1.284	0.138	1.116	1.608
500	12.869	0.275	12.329	13.326
1000	84.176	3.381	79.919	90.101
2000	636.942	13.970	618.909	660.740

7.1.4 Speedup en Maquina 1

N	OpenMP 4h	OpenMP 8h
100	0.34x	0.36x
500	3.11x	4.15x
1000	3.39x	4.92x
2000	3.57x	5.41x

7.2 Maquina 2 - Resultados de Benchmark

7.2.1 Serial

N	Promedio (ms)	Desvio (ms)	Min (ms)	Max (ms)
100	0.359	0.011	0.336	0.368
500	38.901	1.105	36.936	40.704
1000	341.735	12.713	317.089	369.756
2000	3776.698	38.891	3734.502	3831.864

7.2.2 OpenMP 4 hilos

N	Promedio (ms)	Desvio (ms)	Min (ms)	Max (ms)
100	0.956	0.268	0.789	1.593
500	12.903	0.706	11.995	14.248
1000	91.95	2.192	89.147	95.55
2000	1040.642	27.877	1013.749	1110.897

7.2.3 OpenMP 8 hilos

N	Promedio (ms)	Desvio (ms)	Min (ms)	Max (ms)
100	0.831	0.157	0.645	1.067
500	11.356	0.838	10.228	12.517
1000	81.392	2.773	76.246	85.358
2000	722.182	16.693	698.952	754.557

7.2.4 Speedup en Maquina 2

N	OpenMP 4h	OpenMP 8h
100	0.38x	0.43x
500	3.01x	3.42x
1000	3.72x	4.20x
2000	3.63x	5.23x

8. Analisis comparativo entre maquinas

8.1 Comparacion de rendimiento serial

N	Maquina 1 (ms)	Maquina 2 (ms)	Diferencia
---	----------------	----------------	------------

100	0.459	0.359	Maq2: 21.8% mas rapida
500	53.411	38.901	Maq2: 27.1% mas rapida
1000	413.872	341.735	Maq2: 17.4% mas rapida
2000	3443.264	3776.698	Maq1: 8.1% mas rapida

La Máquina 2 es generalmente más rápida en casos pequeños a medianos, pero en N=2000 la Máquina 1 logra mejor rendimiento.

8.2 Comparacion de OpenMP 4 hilos

N	Maquina 1 (ms)	Maquina 2 (ms)	Diferencia
100	1.350	0.956	Maq2: 29.2% mas rapida
500	17.160	12.903	Maq2: 24.8% mas rapida
1000	122.030	91.95	Maq2: 24.6% mas rapida
2000	964.856	1040.642	Maq1: 7.3% mas rapida

8.3 Comparacion de OpenMP 8 hilos

N	Maquina 1 (ms)	Maquina 2 (ms)	Diferencia
100	1.284	0.831	Maq2: 35.3% mas rapida
500	12.869	11.356	Maq2: 11.7% mas rapida
1000	84.176	81.392	Maq2: 3.3% mas rapida
2000	636.942	722.182	Maq1: 11.7% mas rapida

8.4 Conclusiones del analisis comparativo

- 1. **Máquina 2 ventaja en casos pequeños y medianos:** Es significativamente más rápida en N=100, N=500 y N=1000 con todas las configuraciones.
- 2. **Máquina 1 ventaja en casos grandes:** En N=2000, Máquina 1 recupera ventaja, especialmente en OpenMP-8, donde es 11.7% más rápida.
- 3. **Escalabilidad de OpenMP:** En ambas máquinas, OpenMP muestra beneficio creciente con el tamaño del problema. OpenMP-8 es consistentemente la configuración más rápida.

4. **Comportamiento consistente:** El patrón de mejora con OpenMP-4 y OpenMP-8 es similar en ambas máquinas, confirmando la solidez del enfoque de paralelización.

9. Perfilado de memoria y CPU en ambas maquinas

9.1 Perfilado Maquina 1

9.1.1 Serial

N	Wall_s	CPU_s	Peak Working Set (MB)	Peak Private Memory (MB)	CPU/Wall
100	0.037696	0.000000	3.02	0.44	0.00
500	0.091642	0.046875	6.61	4.36	0.51
1000	0.464862	0.437500	18.86	15.82	0.94
2000	3.597797	3.515625	64.94	61.69	0.98

9.1.2 OpenMP 4 hilos

N	Wall_s	CPU_s	Peak Working Set (MB)	Peak Private Memory (MB)	CPU/Wall
100	0.039136	0.000000	3.02	0.44	0.00
500	0.056573	0.062500	8.91	5.78	1.10
1000	0.178938	0.468750	23.89	20.13	2.62
2000	1.056642	3.750000	81.19	77.48	3.55

9.1.3 OpenMP 8 hilos

N	Wall_s	CPU_s	Peak Working Set (MB)	Peak Private Memory (MB)	CPU/Wall
100	0.039834	0.000000	3.02	0.44	0.00
500	0.057444	0.125000	8.90	5.91	2.18
1000	0.146524	0.703125	24.02	20.27	4.80
2000	0.767008	5.109375	81.09	77.71	6.66

9.2 Perfilado Maquina 2

9.2.1 Serial

N	Wall_s	CPU_s	Peak Working Set (MB)	Peak Private Memory (MB)	CPU/Wall
100	0.045022	0.015625	3.03	0.43	0.35
500	0.085740	0.031250	6.27	4.38	0.36
1000	0.379463	0.343750	19.15	15.84	0.91
2000	3.696946	3.531250	64.96	61.71	0.96

9.2.2 OpenMP 4 hilos

N	Wall_s	CPU_s	Peak Working Set (MB)	Peak Private Memory (MB)	CPU/Wall
100	0.047154	0.015625	3.04	0.43	0.33
500	0.069492	0.031250	9.14	5.64	0.45
1000	0.177312	0.390625	22.81	19.99	2.20
2000	1.193069	4.265625	80.76	77.31	3.58

9.2.3 OpenMP 8 hilos

N	Wall_s	CPU_s	Peak Working Set (MB)	Peak Private Memory (MB)	CPU/Wall
100	0.054134	0.046875	3.03	0.43	0.87
500	0.065148	0.125000	9.46	5.77	1.92
1000	0.152555	0.609375	20.92	20.11	3.99
2000	0.858713	5.359375	80.94	77.44	6.24

9.3 Analisis del perfilado

- ****Memoria privada****: Ambas máquinas muestran el mismo comportamiento cuadrático en memoria privada, confirmando la estimación teórica de $16N^2$ bytes.
- ****CPU vs Wall****: En configuraciones con 8 hilos, la razón CPU/Wall es mayor (4.80 a 6.66), indicando uso paralelo efectivo.
- ****Consistencia entre máquinas****: Los patrones de perfilado son muy similares entre máquinas, validando la portabilidad y confiabilidad del código.

10. Conclusiones

10.1 Validacion del paralelismo

1. **OpenMP es efectivo:** La versión con OpenMP reduce tiempos de ejecución desde 3.01x hasta 5.41x dependiendo de la configuración.
2. **Overhead de OpenMP:** Para $N=100$, el overhead es dominante. Desde $N=500$ en adelante, el beneficio supera ampliamente la sobrecarga.
3. **Escalabilidad:** OpenMP-8 supera a OpenMP-4 en todos los casos no triviales, demostrando que el problema se comporta bien con más hilos.

10.2 Caracterizacion de hardware

1. **Máquinas similares en capacidad:** Ambas máquinas tienen comportamiento parecido, con variaciones dentro del 35% en el peor caso.
2. **Problema escalable:** El crecimiento cubico del tiempo con tamaño de matriz es consistente en ambas máquinas.
3. **Memoria no es cuello de botella:** La razón CPU/Wall cercana a 1 (serial) y > 4 (OpenMP-8) indica que el problema está dominado por CPU, no por memoria.

10.3 Recomendaciones

1. Para $N \leq 100$: usar versión serial (menor sobrecarga).
2. Para $N \geq 500$: usar OpenMP con 8 hilos en estas máquinas.
3. La paralelización es robusta y mejora con máquinas más poderosas.

11. Referencias

- [Código Serial](./src/serial/multmat_serial.c)
- [Código OpenMP](./src/openmp/multmat_openmp.c)
- [Benchmark Summary](./benchmark_summary.csv)
- [Profile Summary](./profile_summary.csv)